

8, 9주 차 보고서

(2026년 2 월 15 일 ~ 2026년 2 월 28 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

```
// Overlapped 모델 (Completion Routine 콜백 기반)
// - 비동기 입출력 함수 완료되면, 쓰레드마다 있는 APC 큐에 일감이 쌓임
// - Alertable Wait 상태로 들어가서 APC 큐 비우기 (콜백 함수)
// 단점) APC 큐 쓰레드마다 있다! Alertable Wait 자체도 조금 부담!
// 단점) 이벤트 방식 소켓 : 이벤트 1:1 대응

// IOCP (Completion Port) 모델
// - APC -> Completion Port (쓰레드마다 있는 건 아니고 1개, 중앙에서 관리하는 APC 큐?)
// - Alertable Wait -> CP 결과 처리를 GetQueuedCompletionStatus
// 쓰레드랑 궁합이 굉장히 좋다!

// CreateIoCompletionPort
// GetQueuedCompletionStatus
```

```

enum IO_TYPE
{
    READ,
    WRITE,
    ACCPET,
    CONNECT,
};

struct OverlappedEx
{
    WSADOVERLAPPED overlapped = {};
    int32 type = 0; // read, write, accept, connect ...
};

void WorkerThreadMain(HANDLE iocpHandle)
{
    while (true)
    {
        DWORD bytesTransferred = 0;
        Session* session = nullptr;
        OverlappedEx* overlappedEx = nullptr;

        bool ret = ::GetQueuedCompletionStatus(iocpHandle, &bytesTransferred,
            (ULONG_PTR*)&session, (LPOVERLAPPED*)&overlappedEx, INFINITE);

        if (ret == FALSE || bytesTransferred == 0)
        {
            // TODO : 연결 끊김
            continue;
        }

        ASSERT_CRASH(overlappedEx->type == IO_TYPE::READ);

        cout << "Recv Data IOCP = " << bytesTransferred << endl;

        WSABUF wsabuf;
        wsabuf.buf = session->recvBuffer;
        wsabuf.len = BUFSIZE;

        DWORD recvLen = 0;
        DWORD flags = 0;
        ::WSARecv(session->socket, &wsabuf, 1, &recvLen, &flags, &overlappedEx->overlapped, NULL);
    }
}

```

```

vector<Session*> sessionManager;

// CP 생성
HANDLE iocpHandle = ::CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0, 0);

// WorkerThreads
for (int32 i = 0; i < 5; i++)
    GThreadManager->Launch([=]() { WorkerThreadMain(iocpHandle); });

// Main Thread = Accpet 담당
while (true)
{
    SOCKADDR_IN clientAddr;
    int32 addrLen = sizeof(clientAddr);

    SOCKET clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
    if (clientSocket == INVALID_SOCKET)
        return 0;

    Session* session = new Session();
    session->socket = clientSocket;
    sessionManager.push_back(session);

    cout << "Client Connected!" << endl;

    // 소켓을 CP에 등록
    ::CreateIoCompletionPort((HANDLE)clientSocket, iocpHandle, /*key*/(ULONG_PTR)session, 0);

    WSABUF wsaBuf;
    wsaBuf.buf = session->recvBuffer;
    wsaBuf.len = BUFSIZE;

    OverlappedEx* overlappedEx = new OverlappedEx();
    overlappedEx->type = IO_TYPE::READ;

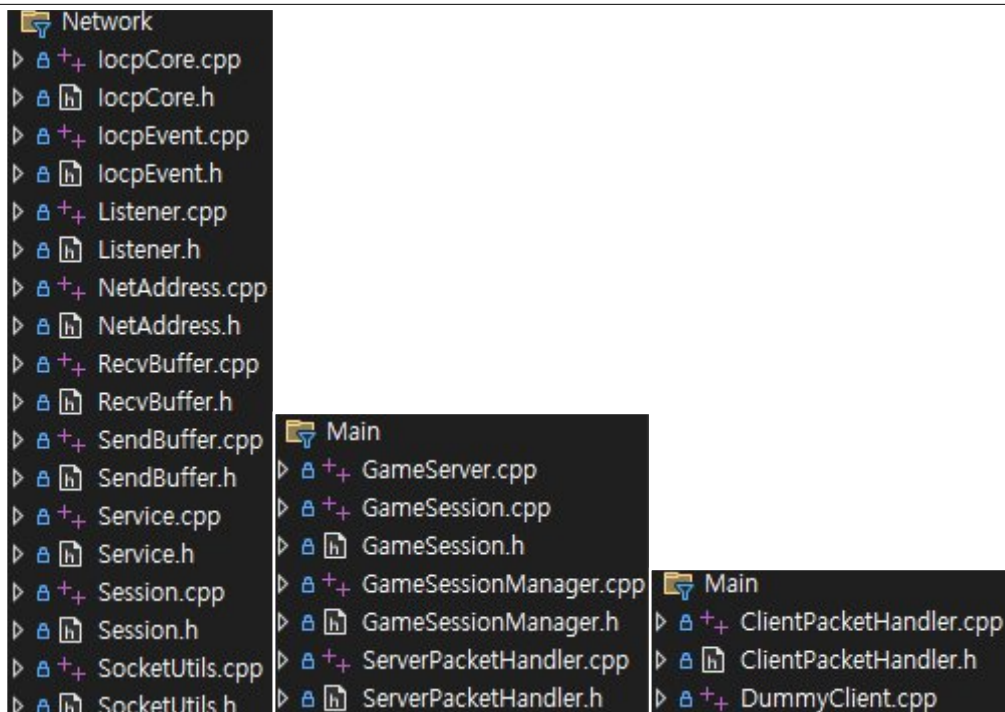
    DWORD recvLen = 0;
    DWORD flags = 0;
    ::WSARecv(clientSocket, &wsaBuf, 1, &recvLen, &flags, &overlappedEx->overlapped, NULL);
}

```

OverlappedEx 구조체는 WSAOVERLAPPED를 확장하기 위해 트릭으로 만든것인데 read인지 write인지 type 정보까지 꺼내 쓰기 위해 사용한다.

CreateIoCompletionPort를 호출해서 os에 I/O완료 통지를 받을 문을 만들고 WorkerThread를 실행하는 쓰레드를 미리 만들어 둔다. Accept로 클라이언트 소켓이 생성되면 이를 기존에 만든 iocpHandle에 등록한다. WSARecv를 호출해서 데이터가 들어오면 알려달라고 OS에 예약한다.

워커쓰레드는 GetQueueCompletionStatus함수에서 쓰레드가 대기 상태에 빠지고 I/O작업이 완료되면 OS가 이 쓰레드 중 하나를 깨운다. session에는 전에 등록한 CompletionKey가 돌아온다. 데이터를 다 처리했다면 다음 데이터를 받기 위해 다시 WSARecv를 호출한다. 이 과정이 반복되면서 끊임 없는 통신이 이루어지는 것이다.



이제 본격적으로 네트워크 라이브러리를 만들어 프레임워크 만들기를 시작했다. 어떤 이벤트가 발생했을 때, 그 일을 누구에게 시킬 것인가를 자동화해주는 식으로 만들었다.

locpcore, locpEvent, Listener, NetAddress, Session, SocketUtils 등의 파일을 만들었고, 각각 역할은 이렇다.

locpEvent

위에서 공부했던 OverlappedEx의 객체 지향 버전으로 OVERLAPPED 구조체를 상속받아서 OS가 읽을 수 있는 형태이면서 동시에 나만 아는 EventType정보를 담고 있다.

locpObject

이벤트가 완료되었을 때 실제 로직을 실행할 주체들의 부모이다. GetHandle()과 Dispatch()를 가진다. Listener는 새로운 손님이 오면 처리하고, Session은 손님이 보낸 데이터가 있으면 처리한다.

locpCore

가장 중요한 부분으로 Register는 소켓을 IOCP에 등록할 때 locpObject 자체를 Completion Key로 등록해버린다. 이렇게 하면 나중에 완료 통지가 올 때 이 소켓이 객체의 것이라는 걸 알 수 있다. Dispatch는 GetQueuedCompletionStatus를 계속 호출하다가 일이 완료되면, locpObject를 찾아 그 객체의 Dispatch()를 호출해준다.

Listener

기존의 accept()는 손님이 올때까지 스레드가 멈춰버리지만, 여기서는 AcceptEx라는 비동기 함수를 사용한다. RegisterAccept는 미리 알바생에서 손님이 오면 알려달라고 예약을 걸어두고, ProcessAccept는 손님이 오면 손님에 대해 화할 세션을 만들고, 중요한건 다시 RegisterAccept를 호출해서 다음 손님을 기다리게 한다.

Service :

서버 또는 클라이언트 전체를 관리하는 단위이다. ServerService는 Listener를 보유하며, 외부의 접속을 받아들이는 역할을 수행한다. ClientService는 서버에 접속을 시도하는 역할을 수행하며, 설정된 수만큼 세션을 생성해 연결을 시도한다. _sessions 셋을 통해 현재 연결된 모든 세션의 생명주기를 관리하며, SessionFactory를 통해 정의한 커스텀 세션 객체를 생성할 수 있게 한다.

RecvBuffer

수신된 데이터를 저장하는 순환 구조 형태의 버퍼이다. _readPos와 _writePos를 두어 데이터가 들어온 만큼 쓰고, 처리한 만큼 읽는다. 데이터가 버퍼 끝에 도달했을 때, 남은 데이터를 앞으로 복사하여 여유 공간을 확보한다. OVERLAPPED 구조체를 상속받아 비동기 작업 완료 시 어떤 이벤트인지 구분하고 관련 데이터를 보관하는 역할을 한다.

SendBuffer

메모리 할당 효율을 높이고 파편화를 방지하게 위해 설계한 청크 기반 메모리 관리 시스템이다.

SendBufferChunk를 크게 하나 할당해서 필요한 만큼 잘라쓰고, 다 쓰면 다시 재활용하는 방식이다. SendBuffer는 실제로 데이터를 담는 인터페이스이다. SendBufferChunk의 특정 영역을 가리키는 포인터를 가지고 있다. 사용이 끝나면 Close()를 호출하여 사용한 실제 크기를 기록한다.

SendBufferChunk는 엄청 큰 크기의 실제 메모리 배열을 들고 있다. 하나의 청크 안에서 여러 개의 SendBuffer가 순차적으로 할당된다. _usedSize변수를 통해 현재 어디까지 사용했는지 관리하며, 다쓰면 Reset()을 통해 재사용된다.

SendBufferManager는 SendBufferChunk들을 리스트로 관리한다. 쓰레드 세이프하게 설계하여 여러 쓰레드가 동시에 메모리를 요청해도 안정하게 청크를 빌려주고 회수한다.

기존의 Session은 단순히 바이트 덩어리를 주고 받았는데 PacketSession을 추가하여 그 데이터가 의미 있는 단위인 패킷으로 조립하는 역할을 하게 했다. 패킷의 전체 크기와 패킷 종류를 패킷 헤더 구조체 안에 넣었다. 그리고 OnRecv를 재정의하여 최소한 헤더만큼은 데이터가 왔는지 확인하고, 헤더를 읽어서 패킷의 전체 크기를 파악하고, 전체 패킷 크기만큼 데이터가 다 왔는지 확인 후 패킷이 완성되면 OnRecvPacket을 호출한다. PacketSession은 OnRecv를 sealed로 막아버리고 대신 OnRecvPacket이라는 가상 함수를 사용한다.

BufferReader와 BufferWriter를 추가하여 메모리 버퍼에 데이터를 직렬화 하거나 역직렬화 할 때 사용하는 유틸리티 도구로 만들었다. 네트워크 패킷을 만들거나 파일을 저장할 때, 여러 데이터를 하나의 바이트 덩어리로 묶고 푸는 역할을 편리하게 도와주는 역할을 한다.

BufferWriter는 특정 메모리 공간에 순차적으로 데이터를 기록하는 역할을 한다. operator<<는 정수나 구조체 같은 데이터를 writer << value; 형태로 쉽게 버퍼에 넣을 수 있다. Reserve<T>()는 데이터를 바로 쓰지 않고, 나중에 값을 채우기 위해 특정 크기만큼의 공간을 미리 확보하고 그 주소를 반환한다. 데이터를 쓰기 전에 FreeSize()를 체크해서 버퍼 오버플로우를 방지한다.

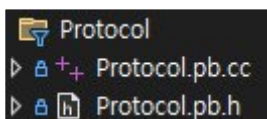
BufferReader는 버퍼에 담긴 바이트 데이터를 다시 의미 있는 데이터 타입으로 읽어오는 역할을 한다. operator>>는 reader >> value; 형태로 버퍼의 데이터를 순서대로 꺼내온다. Peek()는 현재 읽는 위치를 옮기지 않고 데이터만 살짝 확인한다. Read()는 데이터를 읽어오면서 동시에 읽는 위치를 뒤로 이동시킨다.

서버와 클라이언트에 PacketHandler를 만들어주었다.

ServerPacketHandler는 서버에서 클라이언트로 데이터를 보낼 패킷을 조립한다. Make_S_TEST함수는 GSendBufferManager를 통해 빈 버퍼를 할당받는다. BufferWriter를 이용해 ID->HP->공격력 순서로 데이터를 밀어 넣는다. 버퍼 리스트의 경우 buffs.size()를 먼저 기록하여 클라이언트가 몇 개를 읽어야 할지 알려준 뒤, 루프를 돌며 세부 정보를 기록한다. 최종적으로 패킷 전체 크기를 헤더에 기록하고 SendBuffer를 반환한다.

ClientPacketHandler는 서버로부터 받은 바이너리 덩어리를 해석한다. HandlePacket은 패킷 헤더를 먼저 읽어 ID를 확인하고, 해당 ID에 맞는 처리함수로 분기한다. Handle_S_TEST는 BufferReader를 사용하여 서버가 넣은 순서 그대로 데이터를 뽑아낸다. buffCount를 읽어온 뒤, vector::resize()를 통해 공간을 확보하고 루프를 돌며 버퍼 정보를 채운다. 그리고 cout해서 수신된 데이터를 화면에 출력하여 확인한다.

현재는 패킷의 종류가 하나뿐이라 switch문이 단순하지만 패킷 종류가 여러 개가 되면 이 방식은 관리가 힘들어진 다. 후에 Google Protocol Buffer를 사용할 예정이라서 일단은 문제가 있는 방식이라도 이렇게 연습해보았다.



Google Protocol Buffers를 프로젝트에 추가해보았다. 구글에서 만든 데이터를 효율적으로 주고받기 위한 규격이다. 보통 JSON이나 XML을 사용하지만 용량이 크고 파싱하는데 에너지가 많이 든다. Protobuf는 이를 해결하기 위해 데이터를 이진 형태로 압축한다.

다음 주 할 일

패킷 자동화와 JobQueue에 대해서 공부를 할 예정이다. 데이터베이스를 연동하는 것도 연습해볼것이다.

특이사항